

Kaitlyn Carillo
Harini Baskar
Ellie Pearson
Siri Gunturi

Lab 5 Chatbot Report

Introduction

kaitlyn-bot is an IRC chatbot that connects to the libera.chat network. It listens for raw IRC messages, parses them, and dispatches appropriate responses. The bot handles three types of behavior: protocol commands, a greeting finite state machine, and a weather query feature. This report will discuss the development and usage of kaitlyn bot in 3 sections. Section 1 will discuss the chatbot system and the main loop, Section 2 will discuss Phase 1 (protocol commands) and Phase 2 (greeting finite state machine). Finally, Section 3 will discuss the weather query that we chose to implement as our extra feature.

Section 1 - Description of chatbot system, main loop, and mechanics

The chatbot system consists of an IRC socket, main event loop, state machine, timer system, and an NLP layer to handle the weather query. Our IRC class wraps a TCP socket and handles low-level protocol communication (ex. NICK, USER, JOIN, PRIVMSG, PING, PONG). The timer system consists of three timestamps that drive the proactive greeting and timeout logic. The state machine consists of a global state variable that tracks the current phase of any active greeting conversation. Based on the messengers, the state variable is changed in compliance with the Phase 2 instructions. For the main loop, we implemented a while True loop that continuously polls for new IRC messages, checks timer thresholds, and dispatches logic based on parsed messages.

Every iteration of the while True loop performs five steps. T1, t2, and t3 are checked to determine if a greeting or timeout action is due. `irc.get_response()` is called to perform a non-blocking `recv()`, returning an empty string on timeout. PING messages are automatically answered with PONG inside that method. The response text is lowercase for matching. The sender's nickname is extracted from the IRC message prefix. Command and greeting checks are run against the parsed message using substring matching and the current state variable. Finally, appropriate action is taken based on the parsed message.

Section 2 - Discussion of implementation phases

Phase 1 established the bot's baseline communication skeleton. The commands "die", "forget", "who are you?", "usage", "hello", and "hi" are supported.

- die : sends farewell message, calls QUIT, exits Python
- forget : clears memory dict, resets all state variables and timers
- who are you? / usage : sends two - line identity response with bot name and capabilities
- hello / hi : responds with a greeting from the `outreach_reply` list

Section 3 - Discussion of extra features

Phase 2 implements a multi-turn greeting conversation following the diagram outlined in the assignment document. The bot supports both the roles of an initiator and a respondent. Three timestamps drive the timer-based escalation and frustration paths. Phrase recognition in Phase 2 is done based on matching a message with specific, hard-coded inquiries.

State	Trigger	Bot Action
START	Initial / after forget	Wait for t1 timer
INITIAL OUTREACH	t1 timer fires, no_response_counter == 0	Sends random greeting to a randomly selected channel user
SECOND OUTREACH	t2 timer fires, no_response_counter == 1	Sends an escalated greeting ("I said HI!", etc.)
GIVE UP FRUSTRATED	t3 timer fires, no_response_counter == 2	Sends frustrated phrase, QUITs and exits
OUTREACH REPLY	Incoming greeting while no_response_counter == 0	Replies with random greeting from outreach_reply list
INQUIRY	Inquiry phrase received in OUTREACH REPLY state	Sends inquiry_reply_2, then inquiry_2 (the "how about you?" turn)
INQUIRY 2	Reply to bot's own initial greeting received	Sends inquiry phrase, awaits reply
END	Final inquiry reply received	Sends inquiry_reply_1, QUITs and exits

Phase 3 implements a weather and location query feature. Kaitlyn-bot can answer two categories of real-time geographic queries addressed to it. It can answer questions about weather and the time in any city in the world. It can also answer questions about population. These queries are detected by keyword matching against weather_words and loc_words sets, and the city name is extracted using spaCy's NER. We used Open-Meteo Geocoding API, Open-Meteo Weather APO, and python zoneinfo. Both APIs are free and don't require an API key.

Service	URL	What It Provides
---------	-----	------------------

Open-Meteo Geocoding API	geocoding-api.open-meteo.com/v1/search	Lat/Ing, timezone, population, country for any city name
Open-Meteo Weather API	api.open-meteo.com/v1/forecast	Current temperature, feels-like, humidity, precipitation probability
Python zoneinfo (stdlib)	Built-in	Converts IANA timezone string to current local datetime